

Advanced C Programming Lab

Session V – Apr 9

Objectives

We will be talking about;

- Local and Global variables (AKA Scope Rules)

In C, variables are only accessible inside the region they are created. This is called scope. A scope in any programming is a region of the program where a defined variable can have its existence and beyond that variable it cannot be accessed

Local Scope

A variable created inside a function belongs to the local scope of that function and can only be used inside that function or the block of code. Local variables are not known to functions outside their own.

Also remember that functions are defined globally. Whether they have global or local variable, they can be used as a function without any issues.

Local Scope

```
#include <stdio.h>
void Function() {
  // Local variable that belongs to Function
  int x = 12;
  // Print the variable x
  printf("%d", x);
}
int main() {
  Function();
  return 0;
}
```

Output:

12

Local Scope

```c
#include <stdio.h>
void Function() {
  // Local variable that belongs to Function
  int x = 12;
}
int main() {
  Function();
  // Print the variable x in the main function
  printf("%d", x);
  return 0;
}
```

Output:

```
[ERROR]
```

Global Variables

To fix this issue, we can use global variables. A variable created outside of a function, is called a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

And, global variables are defined outside a function, usually on top of the program and hold their values throughout the lifetime of your program.

Global Variables

```c
#include <stdio.h>
//Global Variable 'x'
int x = 7;
void myfunc() {
  //We can use 'x' here
  printf("%d\n", x + 1);
}
int main() {
  myfunc();
  //We can also use 'x' here
  printf("%d\n", x);
  return 0;
}
```

Output:

```
8
7
```

Global Variables

When naming variables, If you operate with the same variable name inside and outside of a function, C will treat them as two separate variables; One available in the global scope (outside the function) and one available in the local scope (inside the function).

However, you should avoid using the same variable name for both globally and locally variables as it can lead to errors and confusion.

In general, you should be careful with global variables, since they can be accessed and modified from any function

Global Variables

```c
#include <stdio.h>
// Global variable x
int x = 10;
void myfunc() {
  // Local variable with the same name as the global variable
  int x = 25;
  printf("%d\n", x); // Refers to the local variable x
}
int main() {
  myfunc();
  printf("%d\n", x); // Refers to the global variable x
  return 0;
}
```

Output:

```
25
10
```

Global Variables

```c
#include <stdio.h>
// Global variable
int x = 10;
void myFunction() {
  x = ++x;  // Increment the GLOBAL value of x by 1
}
int main() {
  myFunction();
  printf("%d\n", x); // Print the global variable x
  return 0;
}
```

Output:

11

Global Variables

If you want to access a global variable when a local variable with same name is also there in the program, then you should use the `extern` keyword.

However, it is not wise or efficient to have too many global variables in a C program, as the memory occupied by these variables is not released till the end of the program.

Global Variables

```
#include <stdio.h>
// Global variable x
int x = 14;
int main(){
   // Local variable x
   int x = 48;{
      extern int x;
      printf("Value of global x is %d\n", x);
   }
   printf("Value of local x is %d\n", x);
   return 0;
}
```

Output:

```
Value of global x is 14
Value of local x is 48
```

Thank you for your time and attention!

References:

- https://www.w3schools.com
- https://www.tutorialspoint.com